Cheatsheet · 15 chapters on one page

Master the Claude cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

133 key ideas | 46 formulas | 60 tables | 28 tips and traps

STEP 1 Foundations

1.1 Meet Claude: Models, Tokens, Projects and Skills

What Claude is, where you can use it, the four current models and what they cost, tokens and the context window, how a chat really works, and how claude.ai Projects and Skills save you from repeating yourself.

KEY IDEAS

- **What Claude is, in plain words** Claude is an AI model made by Anthropic. You write to it in normal language and it writes back. It can read and write text, look at images and PDFs, write and run code, and use tools you connect to it.
- **The Claude models and when to pick each** Claude comes in a family of models. Bigger models think more deeply. Smaller ones are faster and cheaper. All four current models read up to 1 million tokens at once and can write up to 128 thousand tokens in one answer.
- **Tokens and the context window** Claude does not read letters or whole words. It reads tokens , which are small pieces of text. A short word is usually one token. A long word may be two or three.
- **How a conversation really works** Claude has no hidden memory between messages. Each time you press send, the app sends the whole conversation so far , and Claude reads all of it again before replying.
- **Your first useful prompt** A good prompt says who you are , what you have , what you want and what shape the answer should take . Here is one a home baker could send on any Sunday.
- **claude.ai Projects: instructions plus knowledge** A Project in claude.ai is a folder for chats about one topic. It has two places to put information:
- **Skills: teach Claude a task once** A Skill is a folder with a file called SKILL.md . The file has a short name and description at the top and the steps for one task below. Claude reads only the name and description at first. When a request matches, it opens the full file.
- **The map of this subject** Each tool and file you will learn has one job. Hover or tap a box to see where it lives.
- **Safe habits from day one**

FORMULAS

What a skill upload looks like

```
            invoice-checker.zip
            └── invoice-checker/
      ├── SKILL.md            ← required: name,
          description, then the steps
      ├── checklist.md        ← optional: extra
       detail Claude reads only when needed
                  └── scripts/
          └── totals.py       ← optional: code
                 Claude can run
```

Lesson: Skills: teach Claude a task once

TABLES

Table: i want to... · use · what it is

I want to...	Use	What it is
chat, write, plan, study	**claude.ai**, the desktop app or the mobile app	A chat window. Add files, use Projects and Skills.
let Claude work inside my files and code	**Claude Code**	An agent that reads, edits and runs things in a folder. Works in the terminal, VS Code, JetBrains, the desktop app and the web.
put Claude inside my own app or script	**Claude API**	Send messages from code and get answers back.

Lesson: What Claude is, in plain words

Table: model · api name · price per million tokens (in / out) · pick it for

Model	API name	Price per million tokens (in / out)	Pick it for
Claude Fable 5.1	`claude-fable-5-1`	$10 / $50	the hardest, longest tasks
Claude Opus 5.5	`claude-opus-5-5`	$4 / $20	most serious work, coding and agents. The usual default.
Claude Sonnet 5.5	`claude-sonnet-5-5`	$2 / $10	everyday work at a good speed
Claude Haiku 5.5	`claude-haiku-5-5`	from $0.10 / $0.50	quick, simple, high volume jobs like sorting emails

Lesson: The Claude models and when to pick each

Table: when you want claude to... · use · chapter

When you want Claude to...	Use	Chapter
do one thing well right now	a clear prompt	2
always know facts about your project	`CLAUDE.md`	5
follow rules only for some files	`.claude/rules/*.md`	6
do a repeatable task the same way each time	`SKILL.md`	7
never do something, with no exceptions	permissions and hooks in `settings.json`	8
hand a side task to a helper	`.claude/agents/*.md`	9
reach a tool or service outside your computer	an MCP server in `.mcp.json`	10

Lesson: The map of this subject

TIPS AND TRAPS

REMEMBER You do not need to be a programmer for the first chapters. Every step names the exact button, file or command.

Lesson: What Claude is, in plain words

COMMON MISTAKE Model names and prices change over time. Before you build something that costs money, check the models page in the Claude docs.

Lesson: The Claude models and when to pick each

1.2 Prompt Engineering That Works

The techniques from Anthropic's own prompting guide, each with a real daily task: clear instructions, reasons, examples, XML tags, roles, output format, thinking, templates and prompt chains.

KEY IDEAS

- **Be clear and direct** Anthropic's own advice is to treat Claude like a brilliant but new employee . It is very capable, but it does not know your customers, your style or what "good" means to you.
- **Say why, not only what** Claude follows a rule better when it knows the reason. A bare rule covers one case. A reason lets Claude handle cases you did not think of.
- **Show examples** Examples are one of the most reliable ways to control format and tone. This is called few shot or multishot prompting.
- **Use XML tags to separate the pieces** A prompt often mixes instructions, background, examples and data. Put each kind in its own tag, such as <instructions> , <context> or <notes> . There are no magic tag names. Pick clear ones and use them the same way every time.
- **Give Claude a role** One sentence about who Claude is for this task changes tone and focus. In the API this goes in the system prompt . In claude.ai it goes in project instructions. In Claude Code the closest place is CLAUDE.md or an output style.
- **Control the format of the answer** Tell Claude the exact shape: a numbered list, a table, JSON, a word limit, a closing line. Showing the shape works even better.
- **Let Claude think before answering** For sums, comparisons and decisions, ask Claude to work through the problem first. Tags make it easy to keep the working separate from the final answer, so a script can read just the answer.
- **Prompt templates for daily work** When you send the same kind of prompt every day, turn it into a template with blanks in double braces. Fill the blanks each time. Later, the same template becomes a skill (chapter 7) or a line in a script (chapter 11).
- **Chain prompts: draft, review, refine** Big jobs go better in steps. The most useful chain is self correction : one prompt drafts, one checks the draft against a list, one rewrites. You can read each step and stop if something is off.
- **A recipe card for any prompt**

FORMULAS

The same task, said clearly

```
Write a 60 word Instagram caption for "Mango
     Lassi Ice Cream", our new flavour.

Reader: college students in Bengaluru who follow
                    our cafe.
 Goal: get them to visit this weekend for the 2
                  for 1 offer.
        Tone: playful, one emoji at most.
End with the line: "Only at Scoops, Indiranagar."
```

Lesson: Be clear and direct

Give the reason, not only the rule

```
      Summarise this lesson for my students.

Some of my students use a screen reader, which
         reads every symbol out loud.
 So please write in full sentences and avoid
          tables, arrows and emoji.
```

Lesson: Say why, not only what

Ask Claude to think first

```
Our shop sold 1,240 items in March and 1,550 in
                    April.
   Returns were 62 in March and 93 in April.

Before you answer, work through it step by step
                inside  tags.
    Then give the final answer inside  tags.

Question: did the return rate go up or down, and
         by how many percentage points?
```

Lesson: Let Claude think before answering

A reusable template: fill the blanks each time

```
You are helping {{name}}, who works as a {{job}}.


                    {{notes}}


    Turn the notes into a status update for
                {{audience}}.
Keep it under {{words}} words and end with one
              clear next step.
```

Lesson: Prompt templates for daily work

TIPS AND TRAPS

REMEMBER Ask for what you **want**, not only what you do not want. "Write in flowing paragraphs" works better than "do not use bullet points".

Lesson: Say why, not only what

COMMON MISTAKE If all your examples look alike, Claude may copy accidental details too, such as always starting with the same word. Vary them.

Lesson: Show examples

REMEMBER Asking Claude to **quote the lines it used** before answering keeps it grounded in your documents instead of its general knowledge.

Lesson: Use XML tags to separate the pieces

1.3 Context Engineering: The Right Information at the Right Time

Why a focused context beats a full one, how to write instructions at the right altitude, just in time retrieval, compaction, progress notes and subagents, and the context commands in Claude Code.

KEY IDEAS

- **Context engineering in one sentence** Prompt engineering is about how you ask . Context engineering is about everything Claude can see when it answers: instructions, files, tools, past messages and notes.
- **Why more is not always better** Claude's attention is a limited budget . Every extra token takes a little of it. As the window fills, the model gets slightly worse at recalling any one fact. Anthropic calls this context rot .
- **Write instructions at the right altitude** A system prompt, CLAUDE.md or SKILL.md can be too low (a long list of brittle if this then that rules) or too high (vague advice that assumes Claude knows your business).
- **Tools and examples are context too** Every tool Claude has comes with a name and description that sit in the context. Good tool sets are small , do not overlap and have clear names . If a human could not say which of two tools fits a job, Claude cannot either.
- **Just in time retrieval** Do not paste everything up front. Give Claude light pointers such as file paths, a list of document titles or a search tool, and let it load details only when a question needs them.
- **Long tasks: compaction, notes and subagents** Some jobs run for hours and outgrow any window. Three techniques keep them on track.
- **Context tools in Claude Code** Claude Code gives you direct control over the window.
- **A context checklist for any task**

FORMULAS

Send a big investigation to a subagent

```
Use a subagent to read every file in
src/payments/ and find where refunds are
calculated.
Report back only: the file names, the function
names, and one line on what each does.
```

Lesson: Long tasks: compaction, notes and subagents

Context commands in Claude Code

```
/context                          # see what is
        using the window right now
/clear                            # start fresh
      when you switch to unrelated work
/compact keep the API changes and the failing
                test names
/btw what was the port number again?     # quick
      side question, not saved to history
```

Lesson: Context tools in Claude Code

TABLES

Table: feature · when it loads · context cost

Feature	When it loads	Context cost
`CLAUDE.md`	session start	every request, full text
Skills	description at start, body when used	low until used
MCP servers	tool names at start, full details on demand	low until a tool is used
Subagents	when started	separate window, only the summary comes back
Hooks	on events, run outside Claude	zero unless the hook returns text

Lesson: Context tools in Claude Code

TIPS AND TRAPS

REMEMBER In Claude Code, run `/context` to see the real numbers for your session. The biggest item is usually files read and old conversation, not your instructions.

Lesson: Why more is not always better

STEP 2 Claude Code and its files

2.1 Claude Code: First Steps

Install Claude Code, run your first session, see how the agent loop works, choose permission modes, follow the explore, plan, code, commit workflow, undo safely, and learn where every Claude Code file lives.

KEY IDEAS

- **What Claude Code is** Claude Code is Claude working as an agent inside a folder on your computer. It can read your files, search them, edit them, run commands such as tests, and use git. You describe the goal in plain words.
- **Install and start, step by step**
- **How it works: the agent loop** For each request, Claude repeats a simple loop: gather context (search and read), take action (edit, run a command), check the result , then decide the next step. It stops when the task is done or it needs you.
- **Permission modes: how much Claude may do alone** The permission mode decides which actions Claude can take without asking. Press Shift+Tab to switch during a session, or set defaultMode in settings.json (chapter 8).
- **Explore, plan, code, commit** Anthropic's recommended workflow for anything bigger than a one line fix has four steps. The first two happen in plan mode , so nothing changes until you agree.
- **Stop, undo and come back later**
- **The commands you will use most** Two more shortcuts: type @ and a file name to point Claude at a file, and press the up arrow for your earlier requests.
- **The .claude folder: where everything lives** Claude Code reads settings and instructions from two places: your project (shared with your team through git) and your home folder (just for you, in every project). The rest of this module fills in each file.

FORMULAS

First requests to try, one at a time

```
what does this project do? give me a 5 line summary
where is the code that calculates the delivery charge?
add a free delivery rule for orders above Rs 499, then run the tests
what files have I changed?
commit my changes with a descriptive message
```

Lesson: Install and start, step by step

Give Claude a way to check its own work

```
def delivery_charge(order_total):
    if order_total >= 499:      # free delivery rule
        return 0
    return 40
```

Lesson: How it works: the agent loop

Plan mode

```
claude --permission-mode plan        # start a session in plan mode
# or press Shift+Tab inside a session until the status bar shows "plan mode on"
```

Lesson: Explore, plan, code, commit

TABLES

Table: mode · what it means · good for

Mode	What it means	Good for
`default` (shown as Manual)	asks the first time each tool is used	learning, unfamiliar projects
`acceptEdits`	edits files and runs simple file commands without asking	trusted work where you review the diff later
`plan`	reads and explores only, no edits to your source	thinking a change through first
`auto`	no routine prompts, a safety check reviews risky actions	long tasks you supervise loosely
`dontAsk`	refuses anything that would need a prompt	locked down scripts
`bypassPermissions`	skips prompts	only inside a throwaway container

Lesson: Permission modes: how much Claude may do alone

Table: command · what it does · chapter

Command	What it does	Chapter
`/init`	writes a starter `CLAUDE.md` for the project	5
`/memory`	opens `CLAUDE.md` files and auto memory	5
`/skills`	lists the skills Claude can use	7
`/permissions`, `/hooks`	view and edit rules and hooks	8
`/agents`	reminds you how to create and manage subagents	9
`/mcp`	manage MCP server connections	10
`/plugin`	browse and install plugins	11
`/model`, `/context`, `/usage`	switch model, see context use, see cost and limits	1, 3
`/doctor`	checks your installation and setup	

Lesson: The commands you will use most

TIPS AND TRAPS

REMEMBER It is not only for programmers. People use it to tidy folders of documents, turn CSV files into reports, rename hundreds of photos and write scripts they could not write themselves.

Lesson: What Claude Code is

COMMON MISTAKE When Claude asks, read the command. "Yes, and don't ask again" saves a rule to `.claude/settings.local.json`, so think before choosing it for commands like `git push`.

Lesson: Permission modes: how much Claude may do alone

COMMON MISTAKE Checkpoints cover edits Claude made with its file tools. They do not undo things outside your folder, such as a database change or a message sent by a command. Use git as well.

Lesson: Stop, undo and come back later

2.2 CLAUDE.md and Memory: What Goes Where

Exactly what to put in CLAUDE.md and what to leave out, the four places it can live and the order they load in, @ imports, CLAUDE.local.md, AGENTS.md, auto memory, and how to create and maintain it with /init, /memory and a prompt audit.

KEY IDEAS

- **What CLAUDE.md is for** CLAUDE.md is a plain Markdown file of instructions Claude Code reads at the start of every session . Write down what you would otherwise explain again and again: how to run the project, the house rules, the traps.
- **Where CLAUDE.md files live** Claude Code loads these from the folder you start in and every folder above it , joined from the top down. A CLAUDE.md inside a subfolder loads later, when Claude reads or edits a file in that subfolder.
- **What to put in, and what to leave out** This table comes from Anthropic's best practices guide. For every line, ask: would removing this cause Claude to make mistakes? If not, cut it.
- **Three real CLAUDE.md files** Each file is short, specific and written for its own audience. Copy the one closest to your work and change the details.
- **Your personal files: ~/.claude/CLAUDE.md and CLAUDE.local.md** Some instructions are about you , not the project. Keep them out of the team file.
- **Split a long file with @ imports** Write @path/to/file anywhere in a CLAUDE.md to pull another file in. Paths are relative to the file that contains the import. Imported files may import others, up to four hops deep. A path inside backticks is not imported.
- **AGENTS.md and other tools** Some teams already have an AGENTS.md file for other coding tools. Claude Code reads it as project instructions only when there is no CLAUDE.md , .claude/CLAUDE.md or CLAUDE.local.md in your folder or above it. If you have both, only the CLAUDE.md files load.
- **Auto memory: notes Claude writes for itself** Besides your CLAUDE.md , Claude Code keeps its own notes per project. When you correct it ("use PowerShell, not bash"), it may save that so you do not have to repeat it next week.
- **Create, check and improve it**

Which files load, and in what order

```
        Started in: /home/asha/work/shop

 1  ~/.claude/CLAUDE.md                      user:
                 short answers
 2  /home/asha/work/CLAUDE.md                work:
                 company rules
 3  /home/asha/work/shop/CLAUDE.md           shop:
             install, test commands
 4  /home/asha/work/shop/CLAUDE.local.md     local:
                 my port is 4100

 Later, only when Claude touches a file in api/:
 5  /home/asha/work/shop/api/CLAUDE.md       api:
             rules for the API folder
```

Lesson: Where CLAUDE.md files live

~/.claude/CLAUDE.md: your personal file for every project

```
                # About me
   - I am Asha, a QA engineer. I prefer short
          answers with the command first.
- My machine is Windows 11 with PowerShell. Give
        PowerShell commands, not bash.

                # Habits
 - Explain any git command that rewrites history
              before running it.
  - When you finish a task, list the files you
                   changed.
```

Lesson: Your personal files: ~/.claude/CLAUDE.md and CLAUDE.local.md

CLAUDE.md with two imports

```
<!-- Maintainers: keep this file under 200 lines.
     Claude never sees this comment. -->
                # GreenLeaf

   See @README.md for what the project does.

               ## Git workflow
                - @docs/git.md

                   ## Note
  Mention a file without loading it by using
       backticks: `@docs/old-notes.md`
```

Lesson: Split a long file with @ imports

CLAUDE.md that reuses an existing AGENTS.md

```
                @AGENTS.md

              ## Claude Code only
   - Use plan mode before changing anything in
                `src/billing/`.
```

Lesson: AGENTS.md and other tools

Create and check a CLAUDE.md

```
             cd ~/projects/greenleaf
                      claude
/init             # Claude studies the project and
           writes a starter CLAUDE.md
 /memory           # open CLAUDE.md files and the
         auto memory folder to edit them
 /context          # check that the file loaded,
              under "Memory files"
```

Lesson: Create, check and improve it

TABLES

Table: scope · file · shared with · put here

Scope	File	Shared with	Put here
Organisation	macOS `/Library/Application Support/ClaudeCode/CLAUDE.md`, Linux `/etc/claude-code/CLAUDE.md`, Windows `C:\Program Files\ClaudeCode\CLAUDE.md`	everyone on managed machines	company standards, security policy
You	`~/.claude/CLAUDE.md`	only you, all projects	your preferences and habits
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	the team, through git	commands, conventions, architecture
You in this project	`./CLAUDE.local.md` (add to `.gitignore`)	only you	your local URLs, test accounts

Lesson: Where CLAUDE.md files live

Table: ✅ include · ❌ leave out

✅ Include	❌ Leave out
commands Claude cannot guess	anything Claude can work out by reading the code
style rules that differ from the usual	standard conventions Claude already knows
how to run tests, which runner you prefer	long API documentation (link to it instead)
branch names and pull request habits	information that changes every week
architecture decisions specific to you	tutorials and long explanations
environment quirks, required variables	a description of every file
gotchas that are not obvious	"write clean code" and other things that go without saying

Lesson: What to put in, and what to leave out

TIPS AND TRAPS

COMMON MISTAKE Claude treats `CLAUDE.md` as **guidance, not enforcement**. It usually follows it, but it can slip. For "must never happen" rules, use a hook (chapter 8).

Lesson: What CLAUDE.md is for

REMEMBER Notice what is missing: no explanation of how Next.js or pandas work, no list of every file. Claude already knows those or can look.

Lesson: Three real CLAUDE.md files

COMMON MISTAKE Imports help you organise, but they do not save context: imported files load at launch too. To load something only for certain files, use a path rule (chapter 6). To load it only for a task, use a skill (chapter 7).

Lesson: Split a long file with @ imports

2.3 Rules: The .claude/rules Folder and Path Rules

Why there is no rules.md, how to build a .claude/rules/ folder step by step, path rules that load only for matching files, ready to copy rules for code, content and contracts, a screenshot guide, rule files for a QA engineer and for a developer, personal and shared rules, and how to choose between rules, CLAUDE.md, skills and hooks.

KEY IDEAS

- **First, the name: there is no rules.md** Many guides and videos talk about a "rules.md" file. Claude Code does not read a file with that name. What it does have is a folder of rule files:

- **Create your first rules, step by step** Claude Code finds every .md file in .claude/rules/ , including subfolders like backend/ . A rule without paths loads at launch with the same priority as .claude/CLAUDE.md .
- **Path rules: load only for matching files** Add YAML front matter with a paths list. The rule then loads only when Claude reads, writes or edits a file that matches one of the patterns. paths is the only field rules use. Any other field is ignored.
- **Ready to copy rule files** Rules are not only for code. Any team that keeps files in a folder can use them.
- **Set up QA and developer rules, with screenshots** This is the real setup of the rule files from the next two parts, in a small Python shop app. Both pictures are real terminal output.
- **Rule files for a QA engineer** Testers repeat the same team standards in every chat: our ID format, no time.sleep , check the status code first. Put each standard in a rule with paths , and it appears exactly when Claude works on that kind of file.
- **Rule files for a developer** Developers need two kinds of rules: a few that apply to all code, and some that matter only in risky places such as database migrations.
- **Personal rules in ~/.claude/rules/** Rules in your home folder apply to every project on your machine. They load before project rules. Neither overrides the other, so if they disagree Claude may follow either. Keep them consistent.
- **Share one set of rules across many projects** The rules folder follows symlinks, so a company can keep its standards in one place and link them into every repository.
- **Rules, CLAUDE.md, skills or hooks?**
- **When a rule does not seem to work**

FORMULAS

.claude/rules/code-style.md (no paths, always loaded)

```
# Code style
- Type hints on every function. `mypy --strict` must pass.
- Name files in snake_case: `order_summary.py`.
- Prefer small pure functions. Keep side effects at the edges.
```

Lesson: Create your first rules, step by step

.claude/rules/backend/api.md (loaded for API files)

```
---
paths:
- "app/api/**/*.py"
---

# API rules
- Validate every request body with the Pydantic models in `app/api/schemas/`.
- Errors use the shape `{ error: { code, message } }` with a 4xx or 5xx status.
- Every new endpoint gets an entry in `docs/openapi.yaml`.
```

Lesson: Path rules: load only for matching files

.claude/rules/testing.md (loaded for test files)

```
---
paths:
- "tests/**/test_*.py"
- "app/**/test_*.py"
---

# Testing rules
- Use pytest. One test file per module, plain English test names.
- No real network calls. Use the fakes in `tests/fakes/`.
- Run only the changed file: `pytest <file> -q`.
```

Lesson: Ready to copy rule files

.claude/rules/dev/python.md (no paths, always loaded)

```
# Python rules for this repo
- Python 3.12. Type hints on every function. `ruff check` and `mypy` must pass.
- Raise our own errors from app/errors.py, never a bare Exception.
- Log with `logger = logging.getLogger(__name__)`. Never print().
- New code needs a test in the matching file under tests/.
```

Lesson: Rule files for a developer

Personal rules for every project

```
~/.claude/rules/
├── preferences.md     # "Answer with the command
      first, then one line of explanation"
└── git.md             # "Never force push.
    Explain any rebase before running it."
```

Lesson: Personal rules in ~/.claude/rules/

Share rules between projects with symlinks

```
# keep company rules in one folder and link them
                into each project
     ln -s ~/company-standards/security.md
           .claude/rules/security.md
ln -s ~/shared-claude-rules .claude/rules/shared
```

Lesson: Share one set of rules across many projects

Check that your rules load

```
/context          # rules that loaded appear in
             the memory files list
/memory             # open and edit rule files
claude --debug    # shows a YAML parse error if a
          rule's frontmatter is broken
```

Lesson: When a rule does not seem to work

TABLES

Table: pattern · matches

Pattern	Matches
`**/*.py`	every Python file, in any folder
`app/**/*`	everything under `app/`
`*.md`	Markdown files in the project root only
`app/**/*.{py,pyi}`	both `.py` and `.pyi` under `app/`

Lesson: Path rules: load only for matching files

Table: what claude is doing · rules loaded

What Claude is doing	Rules loaded
Session starts	`code-style.md`, `security.md` (no `paths`)
Edits `app/api/orders.py`	also `backend/api.md`
Edits `tests/test_cart.py`	also `testing.md`
Edits `README.md`	nothing new
Edits `content/blog/diwali-sale.md`	also `content/blog.md`

Lesson: Path rules: load only for matching files

Table: qa task today · file claude touches · rules that load

QA task today	File Claude touches	Rules that load
Write test cases for a story	`testcases/SHOP-142.md`	`test-design.md`
Automate a checkout API test	`tests/api/test_checkout_api.py`	`test-design.md`, `api-tests.md`
Fix a flaky login UI test	`tests/ui/test_login.py`	`test-design.md`, `ui-tests.md`
Log a bug	`bugs/coupon-not-applied.md`	`bug-reports.md`

Lesson: Rule files for a QA engineer

Table: developer task today · file claude touches · rules that load

Developer task today	File Claude touches	Rules that load
Any task	(session start)	`python.md`, `pull-requests.md`
Add a `gst_number` column to orders	`app/models/order.py`	also `db-migrations.md`
Fix a pricing bug	`app/cart/pricing.py`	nothing new
Add a test for it	`tests/test_pricing.py`	also `qa/test-design.md`

Lesson: Rule files for a developer

Table: you want... · use · loads

You want...	Use	Loads
a few facts for every session	`CLAUDE.md`	every session
a topic kept in its own file, for everyone	rule without `paths`	every session
guidance only for certain files	rule with `paths`	when a matching file is touched
a multi step procedure, run sometimes	skill	description always, steps when used
something that must never happen	hook or permission rule	runs as code, outside Claude

Lesson: Rules, CLAUDE.md, skills or hooks?

TIPS AND TRAPS

REMEMBER Developers and testers share one repo, so they share one rules folder. A developer adding a test gets the QA naming rules too, and a tester never pays the token cost of the migration rule.

Lesson: Rule files for a developer

COMMON MISTAKE A path rule joins the conversation when its file is touched, so `/compact` summarises it away with everything else. It loads again the next time Claude touches a matching file. If a rule must survive compaction, remove its `paths` or move it to the root `CLAUDE.md`.

Lesson: Rules, CLAUDE.md, skills or hooks?

2.4 Skills: Build SKILL.md Step by Step

Build a skill from an empty folder with a screenshot for every step, learn where skills live and how they are named, every front matter field, arguments and live command output, supporting files and scripts, who can start a skill, ready to copy skills for a QA engineer's day and a developer's day, and how to upload a skill to claude.ai.

KEY IDEAS

- **What a skill is** A skill is a folder with a file called SKILL.md . The top of the file has a short description of when to use it. The rest has the steps . Claude Code, claude.ai and the API all use this format.
- **Build your first skill in three steps** The line !`git diff HEAD` is dynamic context : Claude Code runs the command first and pastes the output into the skill, so Claude reads your real changes.
- **Set up a skill in Claude Code, with screenshots** Here is the whole setup for a real QA skill, pick-regression , one screen at a time. The terminal pictures are real output from a real run. Use the numbers or the Back and Next buttons to move between steps.
- **Where skills live and how they get their name**
- **The front matter: what goes at the top** Every field is optional, but description matters most: Claude reads it to decide when to use the skill. Put the main use first and include the words people actually type.
- **Arguments and live data** Text typed after the skill name becomes its arguments. Use $ARGUMENTS for all of it, $0 , $1 for each word (counting from zero), or names listed in arguments . Lines with !`command` are replaced by the command's output before Claude sees the skill.

- **Supporting files and scripts** Keep SKILL.md short (under 500 lines) and put detail in other files in the same folder. Link to them from SKILL.md so Claude knows when to open them. Scripts are run , not read, so a long script costs almost nothing in context.
- **Who can start a skill** Use disable-model-invocation for anything with side effects, like deploying or sending a message. You do not want Claude deciding to deploy because the code "looks ready".
- **Skills for daily work** The best first skills are jobs you already do every week. Here are two more to copy.
- **Skills for a QA engineer's day** A tester repeats the same kinds of work every day: reading stories, writing test cases, logging bugs and deciding what to retest. Each of those is a perfect skill, because the steps are the same every time and only the input changes.
- **Skills for a developer's day** A developer's day has its own loop: pick up an issue, fix it, review the change, ship it. Three skills cover that loop.
- **Skills in claude.ai and the API** The same folder can be uploaded to claude.ai as a ZIP, or sent through the Skills API. Uploads follow the open Agent Skills format, which is stricter than Claude Code:
- **Make skills that work every time**

FORMULAS

Create and try it

```
mkdir -p ~/.claude/skills/summarize-changes
# save the file above as
~/.claude/skills/summarize-changes/SKILL.md
claude
> what did I change?              # Claude picks
the skill because the description matches
> /summarize-changes              # or run it
yourself by name
```

Lesson: Build your first skill in three steps

.claude/skills/invoice-checker/checklist.md

```
# Invoice checklist
- Supplier GSTIN present and 15 characters long
- Invoice number and date present
- Our PO number quoted (format PO-2026-xxxx)
- Payment terms are 30 days or more
```

Lesson: Supporting files and scripts

TABLES

Table: level · when it loads · size

Level	When it loads	Size
1. name and description	always, at start	about 100 tokens per skill
2. the body of SKILL.md	when the skill is used	keep it under 5,000 tokens
3. other files and scripts	only if the steps point to them	scripts run, only their output is read

Lesson: What a skill is

Table: where · path · who gets it

Where	Path	Who gets it
Personal	~/.claude/skills/<name>/SKILL.md	you, in every project
Project	.claude/skills/<name>/SKILL.md	everyone who clones the repo
Plugin	<plugin>/skills/<name>/SKILL.md	anyone who installs it, as /plugin:name
Organisation	the managed settings folder	everyone on managed machines

Lesson: Where skills live and how they get their name

Table: field · what it does

Field	What it does
`name`	the command name. Defaults to the folder name.
`description`	what it does and when to use it. With `when_to_use`, cut at 1,536 characters in the listing.
`when_to_use`	extra trigger phrases or example requests
`argument-hint`, `arguments`	hint shown while typing, and names for each argument
`disable-model-invocation`	`true`: only you can start it
`user-invocable`	`false`: only Claude can use it, hidden from the menu
`allowed-tools`	tools Claude may use without asking while the skill runs
`disallowed-tools`	tools removed while the skill is active
`model`, `effort`	model and effort level while the skill is active
`context`, `agent`	`context: fork` runs it in a subagent of the given type
`paths`	only use it automatically for matching files
`hooks`, `shell`	hooks while it runs, and `bash` or `powershell` for its commands

Lesson: The front matter: what goes at the top

Table: placeholder · becomes

Placeholder	Becomes
`$ARGUMENTS`	everything typed after the name
`$0`, `$ARGUMENTS[0]`	the first word (use quotes for several words)
`$person`	a named argument from `arguments: [person, day]`
`${CLAUDE_SKILL_DIR}`	the skill's own folder, for scripts
`${CLAUDE_PROJECT_DIR}`	the project root

Lesson: Arguments and live data

Table: front matter · you can start it · claude can start it · in context before use

Front matter	You can start it	Claude can start it	In context before use
(default)	yes	yes	the description
`disable-model-invocation: true`	yes	no	nothing
`user-invocable: false`	no	yes	the description

Lesson: Who can start a skill

Table: time · daily task · skill · you type

Time	Daily task	Skill	You type
10:00	Grooming: a new story arrives	`test-cases`	`/test-cases SHOP-142` and paste the story
12:30	Testing: something breaks	`bug-report`	"I found a bug" and paste your notes or the log
16:00	A developer's branch is ready	`pick-regression`	"which tests should I run for this branch?"

Lesson: Skills for a QA engineer's day

Table: time · daily task · skill · who can start it

Time	Daily task	Skill	Who can start it
09:30	Pick up a bug ticket	`fix-issue`	only you, with `/fix-issue 214`
15:00	Check your branch before opening a PR	`pr-review`	you or Claude
Friday	Write the notes for a release	`release-notes`	you or Claude

Lesson: Skills for a developer's day

Table: skill · front matter that matters · why

Skill	Front matter that matters	Why
`fix-issue`	`disable-model-invocation: true`	it edits code, so only you start it
`pr-review`	`context: fork`, `agent: Explore`	a fresh, read only reviewer
`release-notes`	`arguments: [from, to]`	clear names instead of `$0` and `$1`

Lesson: Skills for a developer's day

TIPS AND TRAPS

REMEMBER Use a skill when you notice you are pasting the same instructions for the third time.

Lesson: What a skill is

REMEMBER If the skill does not show up, check three things: the file is named `SKILL.md` in capitals, the front matter starts on line 1 with `---`, and the folder is inside `.claude/skills/`. Run `/skills` to see every skill Claude Code found.

Lesson: Set up a skill in Claude Code, with screenshots

COMMON MISTAKE `allowed-tools` skips permission prompts while the skill runs. Read the `allowed-tools` of skills in a repository before you trust it.

Lesson: Who can start a skill

REMEMBER Pair these skills with the QA rules from Chapter 6. The skill says **what to do**. The rule in `.claude/rules/qa/` says **how your team does it**, such as the test case ID format, and loads only when Claude touches test files.

Lesson: Skills for a QA engineer's day

2.5 settings.json, Permissions and Hooks

The five settings levels and which one wins, allow, ask and deny rules and how they are matched, ready to copy settings files for the team, for you and for your machine, and four real hooks that block commands, protect files, add live facts and log edits, each run with the JSON Claude Code sends.

KEY IDEAS

- **The settings files and which one wins** Settings are plain JSON files. Each one has a different reach. When two files set the same key, the higher one in this list wins. Lists such as permission rules are combined from all files.
- **Permission rules: allow, ask and deny** A rule names a tool, and optionally what it may touch: Bash(pytest *) , Read(./.env) , Edit(app/**) , WebFetch(domain:example.com) . A * matches anything.
- **Ready to copy settings files** A safety detail: defaultMode values auto and bypassPermissions are ignored in project files, so a repository cannot switch your safety checks off. Allow rules in a project file apply only after you trust the folder. Deny and ask rules apply straight away.
- **Hooks: code that runs on events** A hook is a command Claude Code runs at a fixed moment, such as just before a tool runs or when Claude is waiting for you. Unlike CLAUDE.md , a hook always runs.
- **Hook 1: block dangerous commands** Claude sees the "Blocked" message and tries a safer way, or asks you.

- **Hook 2: protect secret files, with a JSON answer** Instead of an exit code, a PreToolUse hook can print JSON with permissionDecision set to deny , ask or allow , plus a reason Claude will read.
- **Hook 3: add live facts to prompts** A UserPromptSubmit hook can add text to Claude's context with additionalContext , nested inside hookSpecificOutput . Here it tells Claude about a release freeze, but only when the prompt is about deploying.
- **Hook 4: after every edit, log, format or notify** Two more from the official guide, ready to paste:
- **Check and debug**

FORMULAS

~/.claude/settings.json (just you, every project)

```
{
  "$schema":
"https://json.schemastore.org/claude-code-
settings.json",
  "model": "claude-opus-5-5",
  "permissions": {
    "defaultMode": "acceptEdits"
  },
  "autoMemoryEnabled": true
}
```

Lesson: Ready to copy settings files

TABLES

Table: priority · file · reaches

Priority	File	Reaches
1 (highest)	managed settings from your organisation	everyone on managed machines, cannot be overridden
2	`--settings` on the command line	this one session
3	`.claude/settings.local.json`	you, in this project (not committed)
4	`.claude/settings.json`	the whole team, through git
5	`~/.claude/settings.json`	you, in every project

Lesson: The settings files and which one wins

Table: command · rules: deny bash(rm *) , ask bash(git push *) , allow bash(pytest *) and bash(git *)

Command	Rules: deny `Bash(rm *)`, ask `Bash(git push *)`, allow `Bash(pytest *)` and `Bash(git *)`
`pytest tests/`	allow
`git status`	allow
`git push origin main`	ask (ask is checked before allow)
`rm -rf build`	deny
`/bin/rm -rf build`	no rule matches, so the permission mode decides
`python manage.py migrate`	no rule matches, so the permission mode decides

Lesson: Permission rules: allow, ask and deny

Table: event · fires · typical use

Event	Fires	Typical use
`SessionStart`	a session begins or resumes	load fresh facts
`UserPromptSubmit`	you send a prompt	add context, block secrets in prompts
`PreToolUse`	before a tool runs, can block it	stop dangerous commands, protect files
`PostToolUse`	after a tool succeeds	format, lint, log
`Notification`	Claude needs your attention	desktop alert
`Stop`	Claude finishes its answer	run checks before it stops
`PreCompact`, `SessionEnd`	before compaction, at the end	save notes, clean up

Lesson: Hooks: code that runs on events

TIPS AND TRAPS

COMMON MISTAKE Settings files are strict JSON. A comment or a trailing comma breaks the file. Add the `$schema` line to get autocomplete and checks in your editor, and run `/status` to confirm what loaded.

Lesson: The settings files and which one wins

COMMON MISTAKE Look at the `/bin/rm` line. Pattern rules match the command as written, so `Bash(rm *)` does not catch `/bin/rm` or `bash -c 'rm ...'`. Use deny rules for clear cases and a **hook** for anything that truly must be stopped.

Lesson: Permission rules: allow, ask and deny

REMEMBER A hook that says `allow` still cannot override a deny rule in settings. Deny always wins.

Lesson: Hook 2: protect secret files, with a JSON answer

COMMON MISTAKE Hooks run with your user's permissions. Read every hook in a repository you clone before you trust the folder.

Lesson: Check and debug

2.6 Subagents and Output Styles

What a subagent is and when to use one, the built-in agents, writing .claude/agents files step by step with every field, ready to copy reviewer, tester and researcher agents, calling and remembering, and output styles that change how Claude talks.

KEY IDEAS

- **What a subagent is** A subagent is a helper that Claude can hand a task to. It has its own context window , its own instructions and its own allowed tools. It does the work, then sends back only a short result . Your main conversation stays small and focused.
- **Built-in subagents you already have** Claude uses these on its own. You will see a row in the transcript such as Explore(Find the payment code) .
- **Create your first subagent, step by step**
- **Every field in an agent file** Multi word fields use camelCase, such as maxTurns . A misspelled field is ignored without an error.
- **Ready to copy agents**
- **Calling a subagent**
- **Agents that remember** With memory set, the agent keeps its own notes folder and gets better over time, for example learning which support issues keep coming back.
- **Output styles: change how Claude talks for a whole session** An output style changes Claude's role, tone and answer format for the session. Four built-in styles sit on top of the Default:

FORMULAS

Ask Claude to write the agent file for you

```
Create a personal code-improver subagent in
~/.claude/agents/ that scans
files and suggests improvements for readability,
performance, and best
practices. It should explain each issue, show the
current code, and
provide an improved version. Make it read-only
and have it use Sonnet.
```

Lesson: Create your first subagent, step by step

.claude/agents/broken.md

```
---
name: team:helper
descripton: Helps the team
max_turns: 10
---

You help the team with anything.
```

Lesson: Every field in an agent file

Run a whole session as one agent

```
claude --agent code-reviewer        # the whole
session runs as that agent
/agents                             # prints a
reminder of where agent files live
```

Lesson: Calling a subagent

Switch output styles

```
/output-style                  # list styles and
see the current one
/output-style concise          # switch to a
built-in style
/output-style "Teach me"       # switch to your
own style
```

Lesson: Output styles: change how Claude talks for a whole session

TABLES

Table: use a subagent when… · stay in the main chat when…

Use a subagent when…	Stay in the main chat when…
a task reads many files or long logs	you need quick back and forth
you want a specialist with fixed rules, like a reviewer	several steps share a lot of context
several independent searches can run in parallel	the change is small and quick

Lesson: What a subagent is

Table: agent · what it does

Agent	What it does
Explore	fast, read only search of a codebase. Starts without your `CLAUDE.md` to stay light.
Plan	read only research used in plan mode
general-purpose	complex tasks that need both exploring and changing
claude	the catch all when nothing more specific fits
statusline-setup, claude-code-guide	small helpers for the status line and for questions about Claude Code

Lesson: Built-in subagents you already have

Table: field · what it does

Field	What it does
`name` (required)	unique id, no `:`
`description` (required)	when Claude should delegate to it. Add "use proactively" to encourage it.
`tools`, `disallowedTools`	allow list or block list of tools. Leave `tools` out to inherit all.
`model`	`sonnet`, `opus`, `haiku`, `fable`, a full model id, or `inherit`
`permissionMode`	`default`, `acceptEdits`, `auto`, `dontAsk`, `bypassPermissions`, `plan`
`maxTurns`	stop after this many steps
`skills`	skills loaded in full at start
`mcpServers`	MCP servers only this agent can use
`memory`	`user`, `project` or `local`: notes it keeps between sessions
`hooks`, `effort`, `background`	hooks while it runs, effort level, run in the background
`isolation: worktree`	work on a separate copy of the repository
`omitClaudeMd`, `color`, `initialPrompt`	skip `CLAUDE.md`, colour in the task list, first prompt when run with `--agent`

Lesson: Every field in an agent file

Table: line · problem · fix

Line	Problem	Fix
`name: team:helper`	a name may not contain `:`	`name: team-helper`
`descripton:`	misspelled, so it is ignored and the agent has no description	`description: ...`
(missing)	without a description Claude never knows when to use it	say when to delegate, in plain words
`max_turns: 10`	unknown field, ignored without an error	`maxTurns: 10`

Lesson: Every field in an agent file

Table: style · what changes

Style	What changes
Proactive	starts work right away and makes reasonable assumptions
Concise	leads with the result, no preamble or recap
Explanatory	adds short "Insight" notes explaining the choices
Learning	explains and leaves small pieces of code for you to write

Lesson: Output styles: change how Claude talks for a whole session

2.7 MCP: Connect and Build Servers

What MCP is, connecting your first server in three steps, scopes and exactly what goes in .mcp.json, building a real expenses server in Python step by step, the JSON that travels between them, using MCP in Claude Code and Claude Desktop, and adding connectors in claude.ai, with a screenshot for every setup step.

KEY IDEAS

- **What MCP is** MCP (Model Context Protocol) is an open standard that lets Claude use tools and data from other programs: your issue tracker, a database, a calendar, or a script you wrote yourself. One program, the MCP server , offers the tools.
- **Connect a server, step by step**

- **Set up MCP in Claude Code, with screenshots** Here is the expenses server from this chapter added to Claude Code for real, then a shared server added for the whole team. The terminal pictures are real output, with the home folder shown as ~ .
- **Scopes and the .mcp.json file** When the same name is in several scopes, local wins, then project, then user. Claude Code asks once before it uses servers from a project's .mcp.json .
- **Build your own server in Python** Here is a server that saves expenses. Claude will be able to say "Saved team lunch for ₹850" because it called your code.
- **What travels between Claude and a server** Under the hood, MCP is a conversation of small JSON messages (JSON-RPC). For a stdio server they go one per line on stdin and stdout. First a handshake, then Claude asks for the tool list, then it calls a tool.
- **Using MCP inside Claude Code** Every MCP tool gets a full name mcp__<server>__<tool> . Use that name in permission rules, in a skill's allowed-tools , in an agent's tools and in hook matchers.
- **The same servers in Claude Desktop** The Claude Desktop chat app reads its own file, claude_desktop_config.json . Open it from Settings, then Developer, then Edit Config. On macOS it is in ~/Library/Application Support/Claude/ and on Windows in %APPDATA%\Claude\ .
- **MCP in claude.ai: connectors, with screenshots** In claude.ai an MCP server is called a connector . Popular services are in the Connectors Directory. Your own server can be added as a custom connector .

FORMULAS

Your first MCP server in three steps

```
# 1. Add the free Claude Code docs server (no login needed)
claude mcp add --transport http claude-code-docs https://code.claude.com/docs/mcp

# 2. Check it connected
claude mcp list

# 3. Use it
claude
> Use the claude-code-docs server to find how hooks block a command
```

Lesson: Connect a server, step by step

Set up a Python server project

```
mkdir expenses-mcp && cd expenses-mcp
python3 -m venv .venv && source .venv/bin/activate
pip install "mcp≥2"
# write server.py (below), test it with try_it.py (below), then:
claude mcp add --transport stdio expenses -- "$PWD/.venv/bin/python" "$PWD/server.py"
```

Lesson: Build your own server in Python

Using tools, resources and prompts

```
Add a ₹450 expense for printer ink, then tell me the total
Compare @github:issue://123 with @docs:file://api/authentication
/mcp__github__pr_review 456
```

Lesson: Using MCP inside Claude Code

claude_desktop_config.json (Claude Desktop app)

```
{
  "mcpServers": {
    "expenses": {
      "command": "/Users/asha/expenses-mcp/.venv/bin/python",
      "args": ["/Users/asha/expenses-mcp/server.py"]
    }
  }
}
```

Lesson: The same servers in Claude Desktop

TABLES

Table: a server can offer · what it is · how you use it

A server can offer	What it is	How you use it
Tools	actions Claude can run, like "add expense"	Claude calls them when they fit your request
Resources	read only data, like a file or a record	type `@server:uri` in your prompt
Prompts	ready made prompt templates	type `/mcp__server__prompt`

Lesson: What MCP is

Table: scope · loads in · shared with team · stored in

Scope	Loads in	Shared with team	Stored in
local (default)	this project only	no	`~/.claude.json`, under this project
project	this project only	yes, through git	`.mcp.json` at the project root
user	all your projects	no	`~/.claude.json`

Lesson: Scopes and the .mcp.json file

Table: fact · what it means for you

Fact	What it means for you
Claude gets your permissions	if you cannot open a file in the source app, Claude cannot either
Free plan: one custom connector	paid plans can add more
Team and Enterprise	an owner enables a connector first, then each person signs in with their own account
Owners set tool permissions	each tool can be **Always allow**, **Needs approval** or **Blocked** for the whole organisation

Lesson: MCP in claude.ai: connectors, with screenshots

TIPS AND TRAPS

REMEMBER If `claude mcp list` shows ✘ **Failed**, run the start command yourself in the terminal. The error you see there is the real reason, most often a wrong path or a missing package.

Lesson: Set up MCP in Claude Code, with screenshots

COMMON MISTAKE Custom connectors are not checked by Anthropic. Only add servers from people and companies you trust, because a connector can read and sometimes change data with your permissions.

Lesson: MCP in claude.ai: connectors, with screenshots

2.8 Daily Automation: Plugins, Scripts and GitHub

Bundle skills, agents, hooks and MCP into a plugin, run Claude inside scripts with claude -p, get JSON back with a schema, answer @claude in GitHub, run reports on a schedule, and five copy and paste recipes for daily work.

KEY IDEAS

- **Plugins: share your whole setup in one folder** By now you have skills, agents, hooks and MCP servers. A plugin puts them in one folder so a whole team can install them with one command. Use plain .claude/ files for one project, and a plugin when several projects or people need the same setup.
- **claude -p: use Claude in scripts** The -p flag runs Claude Code once, prints the answer and exits. That makes it a normal command you can put in shell scripts, Makefiles and scheduled jobs.
- **Get data back, not just text** Ask for JSON in a fixed shape with --json-schema . The data arrives in the structured_output field, ready for the next step of your script.
- **Claude in GitHub Actions**
- **Run it on a schedule** With a prompt input, the action runs on any event, including a timer. GitHub schedules use cron lines in UTC.
- **Daily automation recipes**

FORMULAS

team-toolkit/.claude-plugin/plugin.json

```
{
  "name": "team-toolkit",
  "description": "Our review agent, release notes skill, format hook and docs MCP server",
  "version": "1.0.0",
  "author": { "name": "Platform team" }
}
```

Lesson: Plugins: share your whole setup in one folder

Repeat something during a session

```
/loop 10m check whether the staging deploy finished and tell me when it does
/loop 1h /release-notes
```

Lesson: Run it on a schedule

TABLES

Table: common mistake · what happens · fix

Common mistake	What happens	Fix
`"name": "broken kit"`	a plugin name may not contain spaces	`"name": "broken-kit"`
`.claude-plugin/skills.md`	never loads, only `plugin.json` goes in that folder	move skills to `skills/<name>/SKILL.md`
`skills/notes/README.md`	the skill folder has no `SKILL.md`, so there is no skill	rename it to `SKILL.md` with front matter

Lesson: Plugins: share your whole setup in one folder

Table: flag · what it does

Flag	What it does
`--allowedTools`	tools it may use without asking, in permission rule syntax
`--permission-mode`	a baseline such as `acceptEdits` or `dontAsk`
`--output-format`	`text`, `json` or `stream-json`
`--json-schema`	the exact shape of data you want back
`--append-system-prompt`	extra instructions on top of the default
`--bare`	skip local hooks, skills, plugins, MCP and CLAUDE.md for repeatable runs

Lesson: claude -p: use Claude in scripts

Table: cron line (minute hour day month weekday) · when it runs

Cron line (minute hour day month weekday)	When it runs
`0 9 * * *`	every day at 09:00 UTC
`30 3 * * 1-5`	every weekday at 03:30 UTC, which is 09:00 in India
`0 17 * * 5`	every Friday at 17:00 UTC
`0 6 1 * *`	on day 1 of every month at 06:00 UTC

Lesson: Run it on a schedule

STEP 3 Building with the API

3.1 The Claude API Step by Step

Get a key and install the SDK, make your first call and read every field of the reply, build a tutor with a system prompt and chat history, choose a model and price a job, stream answers, get JSON in a fixed shape, use effort and prompt caching, and handle errors.

KEY IDEAS

- **Get a key and install the SDK** The Claude API lets your own program talk to Claude. You pay per token used, separate from a Claude.ai subscription.
- **Your first call, line by line**
- **System prompts and conversations** The system prompt sets the role and rules for the whole chat. The API does not remember anything between calls, so your code keeps the history and sends it every time.
- **Choose a model and know the cost** All four have a 1 million token context window and up to 128K output tokens. Haiku 5.5 costs $0.50 / $2.50 for prompts over 100K tokens.
- **Streaming: show words as they arrive** Long answers take time. Streaming sends the text piece by piece, so your app can show it at once. Use it for anything long, because it also avoids request timeouts.
- **Get JSON your code can trust** With output_config.format and a JSON schema, the reply is valid JSON in exactly that shape. No more "please answer only in JSON" and hoping.
- **Effort and prompt caching** The stand-in API does not cache, so it reports 0 for both. Against the real API the first call writes the policy to the cache and calls in the next 5 minutes read it.
- **When things go wrong**

FORMULAS

Get a key and install the SDK

```
    # 1. Create a key at platform.claude.com
          (Settings, then API keys)
# 2. Keep it in an environment variable, never in
                    code
export ANTHROPIC_API_KEY="sk-ant-..."        #
                macOS / Linux
setx ANTHROPIC_API_KEY "sk-ant-..."          #
     Windows (open a new terminal after)

          # 3. Install the Python SDK
pip install anthropic                        #
                    Python
```

Lesson: Get a key and install the SDK

Choose a model and know the cost

Exam favourite

$$\text{cost} = (\text{input tokens} \times \text{input price} + \text{output tokens} \times \text{output price}) \div 1{,}000{,}000$$

Lesson: Choose a model and know the cost

TABLES

Table: part · what it means

Part	What it means
`model`	which Claude to use
`max_tokens`	the most it may write. A token is about three quarters of an English word.
`messages`	the conversation so far, as a list of user and assistant turns
`response.content`	a list of blocks. Check `block.type` before reading `.text`.
`stop_reason`	`end_turn` (finished), `max_tokens` (cut off), `tool_use`, or `refusal`
`usage`	tokens in and out, which is what you pay for

Lesson: Your first call, line by line

Table: model · id · $ per million in / out · good for

Model	Id	$ per million in / out	Good for
Fable 5.1	`claude-fable-5-1`	10 / 50	the hardest, longest tasks
Opus 5.5	`claude-opus-5-5`	4 / 20	complex agents, coding, deep analysis
Sonnet 5.5	`claude-sonnet-5-5`	2 / 10	most apps: strong and quick
Haiku 5.5	`claude-haiku-5-5`	0.10 / 0.50	sorting, extracting, high volume

Lesson: Choose a model and know the cost

Table: model · per day · per month

Model	Per day	Per month
Fable 5.1	$50.00	$1,500
Opus 5.5	$20.00	$600
Sonnet 5.5	$10.00	$300
Haiku 5.5	$0.50	$15

Lesson: Choose a model and know the cost

Table: code · meaning · what to do

Code	Meaning	What to do
400	the request is wrong	read the message, fix the field
401	bad or missing key	check `ANTHROPIC_API_KEY`
429	rate limit	the SDK retries. Slow down if it keeps happening.
500, 529	server busy or error	the SDK retries. Try again later.

Lesson: When things go wrong

3.2 Build Your Own Agents

What an agent is, how to define tools, a shopkeeper's stock agent built by hand with the loop shown step by step, the same agent with the tool runner and a spending limit, the Claude Agent SDK, and how to choose the simplest approach that works.

KEY IDEAS

- **What an agent really is** An agent is Claude plus tools plus a loop . Claude reads the goal, asks for a tool, your code runs it and sends back the result, and this repeats until Claude says it is done.
- **Define tools Claude can use** A tool is a name, a description and an input schema . Claude chooses tools by reading the descriptions, so write them for a new colleague: what it does, when to use it and what each input means.
- **Build the loop by hand, step by step** Here is a shopkeeper's assistant. It checks the rice stock, sees it is low and places an order, in two rounds of tool use.
- **Less code with the tool runner** The SDK's tool runner writes the loop for you. Decorate a normal Python function with @beta_tool : the type hints become the schema and the docstring becomes the description.
- **The Claude Agent SDK: Claude Code as a library** The Agent SDK gives your program the same tools, loop, permissions, hooks, skills and MCP support as Claude Code. You do not write tools for reading files or running commands. You just say which ones are allowed.
- **Which way to build your agent**

FORMULAS

Install the Claude Agent SDK

```
mkdir shop-agent && cd shop-agent
python3 -m venv .venv && source
.venv/bin/activate
pip install claude-agent-sdk
export ANTHROPIC_API_KEY="sk-ant-..."
python agent.py
```

Lesson: The Claude Agent SDK: Claude Code as a library

TABLES

Table: good · weak

Good	Weak
`check_stock`: "Look up how many bags... Use before ordering."	`tool1`: "stock"
one clear job per tool	one tool with a `mode` field that does five things
results that say what happened, in words	a bare `true`

Lesson: Define tools Claude can use

Table: tools allowed · what the agent can do

Tools allowed	What the agent can do
`Read`, `Glob`, `Grep`	look but not touch
`Read`, `Edit`, `Glob`	read and change files
plus `Bash`	run commands and tests too

Lesson: The Claude Agent SDK: Claude Code as a library

Table: you want · use

You want	Use
one answer, no tools	a single `messages.create` call (chapter 12)
your own tools, full control of each step	the hand written loop
your own tools, less code	the tool runner
files, commands, web, skills, MCP, ready made	the Claude Agent SDK
Anthropic to run the loop and the sandbox for you	Managed Agents on the Claude API
help inside your own coding sessions	Claude Code subagents (chapter 9)

Lesson: Which way to build your agent

STEP 4 Capstone

4.1 Capstone: Which File Holds What

The full map of Claude files and exactly what goes in each, the real names behind rules.md, skills.md, agents.md and the "prompt file", a table that places any instruction, a complete project reviewed, a homework helper a child can build, and a six week path to the next level.

KEY IDEAS

- **Every file, and what goes in it**
- **The real names behind the names people use**

- **Decide where an instruction belongs** Ask four questions in order. Must it happen every time? Use a hook. Is it about what Claude may do? Use permissions. Is it a procedure or only for some files? Use a skill or a path rule. Otherwise it is a short line in CLAUDE.md .
- **A complete project, reviewed** Here is a whole shop project with every kind of Claude file. Read the tree, then the three main files, and look for what a careful reviewer would flag.
- **Build a homework helper, step by step** This is small enough for a child and a parent to build together in 15 minutes.
- **Your next level, one week at a time**

FORMULAS

greenleaf-shop/.mcp.json (with a mistake)

```
{
  "mcpServers": {
    "claude-code-docs": { "type": "http", "url":
      "https://code.claude.com/docs/mcp" },
    "payments": {
      "type": "http",
      "url": "https://pay.example.com/mcp",
      "headers": { "Authorization": "Bearer
        sk_live_abcdef123456" }
    }
  }
}
```

Lesson: A complete project, reviewed

Start the helper

```
mkdir homework-helper && cd homework-helper
# save CLAUDE.md and the quiz-me skill from this
    page into the folder
claude
> /output-style learning
> /quiz-me fractions
```

Lesson: Build a homework helper, step by step

TABLES

Table: file · put in it · keep out of it

File	Put in it	Keep out of it
`CLAUDE.md`	build and test commands, code style, project facts, "IMPORTANT" rules	long procedures, secrets, things a linter already enforces
`CLAUDE.local.md`	your ports, test accounts, personal shortcuts	anything the team needs
`.claude/rules/*.md`	rules for one area, with `paths:`	rules for every file (those go in CLAUDE.md)
`.claude/skills/<name>/SKILL.md`	a repeatable procedure, with scripts and templates beside it	always-on facts
`.claude/agents/<name>.md`	a specialist: description, tools, model, system prompt	project facts (it reads CLAUDE.md already)
`.claude/settings.json`	permissions, hooks, model, env	instructions in words
`.mcp.json`	MCP servers, with `${VARS}` for keys	real keys
output style	tone, role and answer format	project rules
`plugin.json`	name, description, version of a bundle	skills or agents (they sit beside it)

Lesson: Every file, and what goes in it

Table: people say · what really exists

People say	What really exists
"rules.md"	a folder: `.claude/rules/`, one Markdown file per topic, optional `paths:`
"skills.md"	`SKILL.md` inside its own folder: `.claude/skills/<name>/SKILL.md`
"agents.md"	one file per agent in `.claude/agents/`. A root `AGENTS.md` is shared instructions, read only when there is no CLAUDE.md
"prompt file"	there is no prompt file Claude reads. Save a reusable prompt as a skill (`/name`), a legacy command in `.claude/commands/`, or an MCP prompt
"mcp.json"	`.mcp.json` at the project root, or `~/.claude.json` for local and user scope
"memory.md"	auto memory's `MEMORY.md` index, which Claude writes itself. Your own memory file is `CLAUDE.md`
"system prompt file"	CLAUDE.md for Claude Code, Project instructions on claude.ai, the `system` field in the API

Lesson: The real names behind the names people use

Table: instruction · where it goes

Instruction	Where it goes
Run tests with pytest, check with ruff check .	`CLAUDE.md`, a short fact the whole team needs every session
Always run ruff format after every edit	`.claude/settings.json` hook, because it must happen every time
Never let Claude run rm -rf or git push --force	`.claude/settings.json` permissions, as a deny rule
API key for the payment sandbox is sk_test_123	an environment variable or a secret store, never a Markdown file
Let Claude read our Jira tickets	`.mcp.json`, an MCP server gives Claude that connection
Files in app/api use Pydantic models for every request	`.claude/rules/api.md` with `paths:`, so it loads only for those files
The monthly invoice report, step by step with totals	`.claude/skills/<name>/SKILL.md`, a procedure that loads when needed
A security reviewer that checks every change	`.claude/agents/<name>.md`, a subagent with its own context
Explain like I am 10 and use cricket examples	`~/.claude/output-styles/<name>.md`, how every answer sounds
My local URL is http://localhost:4100	`CLAUDE.local.md`, your private notes for this project
Prices are stored in paise as integers	`CLAUDE.md`, a rule everyone needs

Lesson: Decide where an instruction belongs

Table: problem · why it matters · fix

Problem	Why it matters	Fix
a real key in `.mcp.json`	the file is committed, so everyone with the repository sees the key	`"Bearer ${PAYMENTS_TOKEN}"`
`.gitignore` does not list `CLAUDE.local.md`	private notes and test logins could be committed	add a `CLAUDE.local.md` line

Lesson: A complete project, reviewed

Table: week · do this · you will have

Week	Do this	You will have
1	Run `/init`, trim CLAUDE.md under 200 lines, add two path rules	Claude that knows your project
2	Turn your most repeated request into a skill	one command for a 10 minute job
3	Add a format hook and deny rules for secrets	guarantees, not hopes
4	Connect one MCP server you use daily	Claude working in your real tools
5	Write a reviewer subagent and a `claude -p` recipe	checks and automation
6	Build a small agent with the API for one business task	your own AI tool

Lesson: Your next level, one week at a time

STEP 5 Certification prep

5.1 Claude 101 and Certification Prep

Everything in Anthropic Academy's free Claude 101 course, lesson by lesson: what Claude is, better prompts, Chat, Cowork and Code, projects, artifacts, skills, memory, connectors, enterprise search and Research, use cases by role, how to earn the certificate of completion, and how to prepare for the proctored Claude Certified Associate exam.

KEY IDEAS

- **What Claude 101 is and how you earn the certificate** Claude 101 is a free online course from Anthropic Academy , hosted on Skilljar. Its own description says it teaches you to "use Claude for everyday work tasks, understand core features, and explore resources for more advanced learning". No coding is needed.
- **Module 1: Meet Claude and get better results** What Claude is. Claude is a large language model. It has read a huge amount of text and writes answers by predicting what text should come next. Two facts follow:
- **Module 1: Chat, Cowork and Code on your desktop** The Claude Desktop app has three ways to work. They share one home screen and message box.
- **Module 2: Projects, artifacts and skills**
- **Memory, past chats and incognito** Claude can remember things across chats, and you stay in control of it.
- **Module 3: Connectors, enterprise search and Research** Connectors let Claude read from and act in the apps you already use, such as Google Drive, Gmail, Slack or your own tools. They are built on MCP, the same standard Chapter 10 teaches.
- **Module 4: Claude in action, role by role** The course shows how different teams use the same features. These examples follow the same idea.
- **Module 4: Other ways to work with Claude** The last module, What's next? , points to more Anthropic Academy courses. Chapters 4 to 13 of this subject cover the same ground step by step.
- **The Claude Certified Associate exam (CCAO-F)** If you want a credential with a pass mark, Anthropic has a proctored exam called Claude Certified Associate, Foundations , code CCAO-F .
- **What each exam domain tests, with one tip each** Chapter 1 has the full model table. Chapters 2 and 3 cover prompting and context in depth.
- **Exam style scenarios, worked through** The exam asks what you would do in a work situation. Here is how to think through three typical ones.
- **A seven day study plan**
- **Cheat sheet: numbers and menu paths**

FORMULAS

A good Research request

```
Research how small clinics in India are using AI
          for appointment reminders.
I need: 5 real examples with sources, what each
             one costs if stated,
and the main privacy rules they had to follow.
          Put the sources at the end.
```

Lesson: Module 3: Connectors, enterprise search and Research

Cheat sheet: numbers and menu paths

Exam favourite

up to 10 ×

Lesson: Cheat sheet: numbers and menu paths

Cheat sheet: numbers and menu paths

15 lines

Lesson: Cheat sheet: numbers and menu paths

Cheat sheet: numbers and menu paths

60 · 120 · USD 99

Lesson: Cheat sheet: numbers and menu paths

Cheat sheet: numbers and menu paths

720 / 1,000

Lesson: Cheat sheet: numbers and menu paths

Cheat sheet: numbers and menu paths

12 months

Lesson: Cheat sheet: numbers and menu paths

TABLES

Table: module · lessons in the official outline · covered on this page in

Module	Lessons in the official outline	Covered on this page in
1. Meet Claude	What is Claude? · Your first conversation with Claude · Getting better results · How you'll work with Claude on your desktop	Part 2 and Part 3
2. Organizing your work and knowledge	Introduction to projects · Creating with artifacts · Working with skills	Part 4
3. Expanding Claude's reach	Connecting your tools · Enterprise search · Research for deep dives	Part 5
4. Putting it all together	Claude in action: use-cases by role · Other ways to work with Claude	Part 6 and Part 7
5. Conclusion and certificate	What's next? · Certificate of completion	this part

Lesson: What Claude 101 is and how you earn the certificate

Table: add this · example

Add this	Example
Who it is for	"for busy shop owners who sell on WhatsApp"
The task	"write a 120 word announcement"
Context and facts	"three features: GST invoices, offline, Rs 199 a month"
Format and tone	"friendly and plain, end with one call to action"
An example, if you have one	"here is last month's announcement we liked"

Lesson: Module 1: Meet Claude and get better results

Table: mode · best for · example

Mode	Best for	Example
Chat	back and forth questions, drafts, ideas	"Rewrite this email to sound warmer."
Cowork	longer tasks with many steps, on your files	"Sort the 40 PDFs in my Invoices folder by month and make a summary sheet."
Code	software work: reading, changing and testing code	"Fix the failing login test."

Lesson: Module 1: Chat, Cowork and Code on your desktop

Table: feature · what it is · use it when

Feature	What it is	Use it when
Project	a workspace with its own chats, **instructions** and **knowledge** files	you keep coming back to one topic and do not want to repeat the background
Artifact	a standalone piece of work Claude makes: a document, slides, a design, a dashboard or a small tool	the result is something you would show someone or keep editing
Skill	a folder of instructions Claude loads only when a task needs it	a task has steps that should be done the same way every time

Lesson: Module 2: Projects, artifacts and skills

Table: feature · what it does · where you control it

Feature	What it does	Where you control it
Memory from chats	saves useful facts as topics while you chat	**Settings**, then **Memory**, then **Generate memory from chats**. Read or delete topics there too.
Search past chats	lets Claude look up earlier conversations (paid plans)	the **Search and reference chats** toggle
Project memory	each project keeps its own memory, separate from other projects	turn off **Use account memory** on a project to keep it fully separate
Incognito chat	not saved to history or memory	the ghost icon at the top of a new chat

Lesson: Memory, past chats and incognito

Table: you need... · use

You need...	Use
facts from your own Drive or email	a connector
one answer from all company tools at once	enterprise search
a deep, cited answer from the web	Research

Lesson: Module 3: Connectors, enterprise search and Research

Table: role · a daily job · features

Role	A daily job	Features
Marketing	a month of social posts in the brand voice	project with the brand guide as knowledge, a brand voice skill
Sales	prepare for a call with a new customer	Research on the company, CRM connector for past deals
HR	answer staff questions about leave policy	project with the policy files, instructions to quote the policy line
Finance	check 40 supplier invoices	Cowork on the invoice folder, an invoice skill with a script
Engineering	fix a bug and add a test	Claude Code
QA	test cases from a user story	a test-cases skill, see Chapter 7
Operations	a weekly status report from many tools	connectors plus a report artifact

Lesson: Module 4: Claude in action, role by role

Table: where · what it is for

Where	What it is for
claude.ai on the web	chat, projects, artifacts, research
Claude Desktop	Chat, Cowork and Code, plus local MCP servers
Claude mobile apps	chat on the go. Research on mobile searches only the web, not your connectors.
Claude Code	software work in the terminal, an editor, the desktop app or the web
The Claude API	Claude inside your own app. It is billed separately from Pro, Max, Team and Enterprise plans.

Lesson: Module 4: Other ways to work with Claude

Table: item · detail from the exam guide (version 1.0, july 2026)

Item	Detail from the exam guide (version 1.0, July 2026)
Questions	60, multiple choice and multiple response. Each question says how many answers to pick.
Time	120 minutes
Pass mark	a scaled score of 720 on a scale of 100 to 1,000
Fee	USD 99
Where	proctored through Pearson VUE, online or at a test centre
Valid for	12 months. On time renewal is a free assessment without a proctor.
Retakes	wait 14 days after the first fail, 30 after the second, 90 after the third. At most four tries in 12 months.

Lesson: The Claude Certified Associate exam (CCAO-F)

Table: domain · what you must be able to do · tip to remember

Domain	What you must be able to do	Tip to remember
Output evaluation (21%)	spot mistakes, made up facts, contradictions and bias. Decide when a human must review.	A specific looking citation is still a claim. Check it against the source.
Workflow integration (16%)	see where Claude fits in a process, and explain what it can and cannot do	Start with one repeated task, not the whole department.
Governance and responsible use (15%)	follow data rules and company AI policy, know unsuitable uses	Remove or mask personal data before you upload when policy says so.
Prompting (14%)	write clear prompts, break big tasks into steps, improve a prompt	Who, task, context, format. Then give feedback.
Product and model selection (12%)	pick chat, a project, Research or an artifact, and pick Haiku, Sonnet or Opus	Simple and high volume: a fast, cheaper model. Hard reasoning: a bigger one.
Configuration (12%)	set up project instructions, knowledge and connectors, and keep them current	Instructions say how. Knowledge holds facts.
Troubleshooting (10%)	find why output is poor and fix the prompt or setup	Change one thing at a time and compare.

Lesson: What each exam domain tests, with one tip each

Table: day · do this · try it in claude

Day	Do this	Try it in Claude
1	Claude 101 Module 1. Chapters 1 and 2 here.	rewrite one vague prompt from your work into a specific one
2	Module 2. Make a study project with the instructions above.	add two files as knowledge and ask questions about them
3	Artifacts and skills. Chapter 7 here.	make one artifact and upload one skill
4	Module 3. Connectors, enterprise search, Research.	run one Research request and check three of its sources
5	Module 4 and 5. Earn the Claude 101 certificate.	write down three uses for your own role
6	Exam domains, scenarios and the quiz on this page.	ask your study project to "quiz me" on Domains 2 and 6
7	Read the official exam guide once. Review mistakes.	redo every quiz question you missed

Lesson: A seven day study plan

Table: remember · value

Remember	Value
Projects on the Free plan	5
Extra project capacity with RAG (paid)	up to 10 ×
Artifact usually made when content is over	15 lines
Custom connectors on the Free plan	1
Exam: questions, minutes, fee	60 · 120 · USD 99
Exam pass mark	720 / 1,000
Credential valid for	12 months

Lesson: Cheat sheet: numbers and menu paths

Table: to do this · go here

To do this	Go here
turn on code execution for skills	Settings › Capabilities
upload a skill	Customize › Skills › + › Create skill › Upload a skill
add a connector	Customize › Connectors
turn on a connector or Research in a chat	+ in the message box
see or edit memory	Settings › Memory
check usage	Settings › Usage

Lesson: Cheat sheet: numbers and menu paths

TIPS AND TRAPS

REMEMBER Chapter 2 of this subject goes much deeper into prompting: examples, XML tags, step by step thinking and prompt chains.

Lesson: Module 1: Meet Claude and get better results

COMMON MISTAKE On Team and Enterprise plans, an owner decides whether memory is available. Memory is not available for organisations with HIPAA, public sector or custom data retention agreements.

Lesson: Memory, past chats and incognito

COMMON MISTAKE Third party guides say registration is open only to people at organisations in the Claude Partner Network, using a work email from that organisation. Rules can change, so read the certifications page on the Anthropic Partner Academy before you plan your date.

Lesson: The Claude Certified Associate exam (CCAO-F)

REMEMBER For every scenario ask yourself three things in order: is it safe and allowed, is the answer checked, and is this the simplest tool that does the job.

Lesson: Exam style scenarios, worked through